PRODUCT REQUIREMENTS DOCUMENT

Aim-Truth Rig

An instrumented target that measures — and then tunes — RoboMaster auto-aim accuracy

Field	Value
Owner / PM / Eng	Solo (full-stack: mech, EE, firmware, CV)
Version	v1.0 — Draft for kickoff review
Date	28 July 2026
Target window	16 weeks (Aug – Nov 2026), season begins in 2 weeks
Hardware budget	$150 – $500 ceiling. $163 committed before the Week 10 gate; $333 worst case (§8)
Status	Pending decisions D1–D4 (§10)

1. Executive summary

The team's auto-aim system has no ground truth. Every tuning decision today is made by watching a robot shoot and forming an impression. This project builds a spinning, encoder-instrumented armor target that knows its own true angle to better than 0.05°, so the aimbot's belief can be subtracted from reality and the residual error measured, decomposed, and attributed.

Three findings from this evaluation change the plan as originally scoped:

- **The real critical path is not hardware — it is time synchronisation.** At the target spin rates that matter, 1 ms of timestamp misalignment fabricates ~1° of apparent aiming error, which is the same magnitude as the error being measured. Ground truth is worthless without sub-millisecond alignment to the perception pipeline. This must be solved first, in hardware, before any spinning target is built (§5).
- **Stage 2 (referee-system hit counting) delivers far less than it appears to.** The referee link runs at roughly 200 ms latency with ~3% packet loss, and the armor module tops out at 20 Hz detection. At 3 rev/s the target rotates 216° in that latency window, so a referee hit event can never be attributed to an impact angle. Stage 2 yields a scalar hit rate and nothing more — and a $2 sheet of carbon witness paper yields better spatial data (§6.3).
- **Stage 4 (auto-tuning) is far cheaper than assumed and should be pulled forward.** Once synchronised logs of camera frames plus encoder truth exist, estimator parameters can be swept offline by replaying the recorded data — no simulator, no motor, no ammunition. Auto-tuning becomes an offline optimisation over a fixed dataset that runs in seconds. This is the single highest-leverage item in the project and it is gated only on Phase 0 (§6.5).

> **Recommendation**
>
> Approve, with a resequenced scope. Build the synchronised logging and time-base layer first (Phase 0), the spinning encoder rig second (Phase 1), and the offline replay tuning harness third — ahead of any live-fire work. Live-fire measurement moves later and starts with witness paper rather than piezo sensors. Piezo impact

> localisation stays in scope but behind an explicit go/no-go bench test in Week 10.
>
> Ship a useful number by Week 6 or park the project. A solo builder on a competition team cannot afford a tool that does not pay for itself inside the season.

2. Problem

2.1 What is actually broken

Auto-aim is a chain of six error sources stacked in series: target detection, state estimation and prediction, latency compensation, ballistic solution, gimbal servo tracking, and projectile dispersion. When a shot misses, the current process cannot tell which link failed. Tuning therefore proceeds by changing a parameter, firing a magazine, and forming a subjective judgement about whether it looked better.

This has three concrete costs. Tuning is not repeatable, so a good configuration found on Tuesday cannot be recovered on Friday. It is not attributable, so effort is spent tuning the filter when the real fault is a 40 ms latency mismatch. And it is expensive, because every experiment consumes ammunition, a charged robot, a safe firing space, and usually a second person.

2.2 Why now

The season starts in two weeks. Every week the rig does not exist is a week of guesswork-based tuning that will have to be redone. Conversely, the rig competes directly with robot build work for a single person's time, so it must return measured value quickly or it becomes a distraction. This tension defines the phasing in §6 and the kill criteria in §11.

2.3 Evidence and assumptions

This PRD is written against an inherited codebase of unknown quality with no logging whatsoever. That is itself a finding: the absence of logs is the reason no accuracy baseline exists, and restoring observability is a prerequisite for the rig rather than a side effect of it. Phase 0 begins with a timeboxed archaeology spike to establish what the inherited stack actually does (§6.1).

3. Users and use cases

User	Job to be done	What success looks like to them
Aimbot developer (primary — you)	Change a prediction parameter and know within minutes whether it helped, and by how much.	A single command produces a signed error plot and three numbers: bias, lag, noise.
Next season's CV lead (secondary)	Inherit a system with a defensible accuracy baseline instead of folklore.	A saved dataset and a regression number they can re-run against.
Team leadership (stakeholder)	Decide whether auto-aim is competition-ready and where to spend effort.	One chart: hit rate vs. target spin rate, with confidence intervals.

Note on the primary user: you are building this for yourself, start to finish, with no handoff. That justifies skipping polish — no GUI, no packaging, no onboarding docs — and spending the saved time on

measurement rigour instead. The one exception is the dataset format, which should be self-describing, because datasets outlive the code that made them.

4. Goals and non-goals

4.1 Goals

- **G1 — Make estimator error observable.** Produce a time-aligned comparison of the aimbot's believed target angle and angular velocity against encoder ground truth, at multiple spin rates, with quantified ground-truth uncertainty.
- **G2 — Decompose that error.** Separate the residual into constant bias, effective latency in milliseconds, and random noise (RMS). Three numbers, not one vague impression.
- **G3 — Make tuning cheap and repeatable.** Reduce the cycle from "I want to try a parameter" to "I have a number" to under ten minutes, with no ammunition consumed.
- **G4 — Establish a live-fire outcome baseline.** Measure hit rate and group dispersion as a function of target spin rate, separating the aimbot's contribution from the launcher's inherent dispersion floor.
- **G5 — Automate parameter search.** Sweep estimator parameters offline against recorded datasets and rank them by a defined error metric.

4.2 Non-goals

Stating these explicitly protects a solo builder from scope creep more than any process will.

- Not a competition-legal device. The rig lives in the lab; it is never on the field.
- Not a general-purpose test range. It measures one thing — tracking a rotating armor plate — and does not attempt translating targets, multi-robot scenarios, or realistic lighting variation in v1.
- Not a replacement for field testing. The rig produces a proxy metric. See risk R6.
- Not a closed-loop online auto-tuner. Parameters are searched offline and applied by a human who reviews the result. Autonomous parameter writes to a competition robot are out of scope.
- Not a gimbal characterisation tool, except insofar as gimbal encoder feedback is logged for free alongside everything else.
- No user interface beyond scripts and plots.

5. The core technical constraint: the time base

This section exists because it drives more design decisions than any other consideration, and because getting it wrong invalidates every measurement the rig produces.

5.1 The arithmetic

The rig measures angular error by comparing two time series: what the aimbot believed at time t, and what the encoder recorded at time t. If the two clocks disagree by Δt, the comparison reports a spurious angular error of ω·Δt, where ω is the target's angular velocity.

Target spin rate	Angular velocity	Apparent error from 1 ms of clock skew	Apparent error from 200 µs of clock skew
0.5 rev/s (outpost-like)	180 °/s	0.18°	0.04°
1.5 rev/s (moderate spin)	540 °/s	0.54°	0.11°
3.0 rev/s (aggressive spin)	1080 °/s	1.08°	0.22°

The angular errors worth measuring in a real aimbot are on the order of 0.5° to 2°. A software timestamp taken in userspace on a Linux host, subject to USB transfer delay and scheduler jitter, is routinely wrong by 1–5 ms. **A naive implementation would therefore report an error that is mostly its own measurement artefact, and tuning against it would actively make the aimbot worse.**

5.2 The requirement

REQ-T1: End-to-end alignment between camera exposure onset and the encoder sample attributed to that exposure shall be within ±200 µs, verified by an independent test, at all supported spin rates.

REQ-T2: Ground-truth angular uncertainty from all sources combined (encoder quantisation, non-linearity, and time-base error) shall be ≤ 0.25° at 3 rev/s — i.e. at most half of the smallest error the system claims to resolve.

5.3 How to meet it

Software timestamping cannot meet REQ-T1 and should not be attempted. The solution is hardware triggering, which the machine-vision cameras used in RoboMaster already support:

- The rig microcontroller generates the frame trigger and drives the camera's opto-isolated trigger input. The MCU latches its own encoder count in the same interrupt that asserts the trigger. Camera exposure and encoder sample then share one hardware event, and the residual error is the camera's trigger-to-exposure delay, which is a fixed, datasheet-specified constant of a few microseconds.
- Alternative if the camera cannot be externally triggered: use the camera's strobe output as an input-capture signal on the MCU. This measures rather than commands the exposure instant, and is equally valid.
- Verification method: mount an LED on the rig that the MCU flashes at a known encoder angle. The camera sees the flash; the recorded frame's attributed angle must match the commanded angle. Any disagreement is the sync error, measured directly rather than assumed.

Decision gate

If the team's camera supports neither external trigger in nor strobe out, the entire measurement premise weakens and the spin-rate range must be capped at roughly 1 rev/s to keep the artefact below the signal. Confirm the camera model before ordering anything else. This is open question Q1.

6. Scope and phasing

Four months, solo, in parallel with a live competition season. The sequencing below front-loads everything that is cheap, indoors, and does not need ammunition, and defers everything that needs a range, a charged robot, and a spare afternoon.

Capacity: this plan does not fit, and that is deliberate

Costed out in the companion workbook, the full scope is roughly 66 solo-days. Sixteen weeks at a realistic in-season rate of 2.5 days per week gives about 40. Dropping Phase 3 entirely recovers 15 days and still leaves a gap.

Closing it requires one of three levers: sustain ~3.3 days per week, drop Phase 3 and cut Phase 2 to a single dispersion-floor session, or extend to 20 weeks. Which lever to pull is a decision for the Week 6 payback gate, when there is evidence, rather than now, when there is only optimism. A plan that fits perfectly on day one has not been costed honestly.

6.1 Phase 0 — Observability and time base (Weeks 1–3)

This phase did not appear in the original stage list and is the largest single risk in the project.

Deliverable: a synchronised recording system that captures, per frame, the camera image, the aimbot's internal estimator state, the gimbal's commanded and measured angles, and a hardware-aligned timestamp — written to a self-describing dataset on disk.

- **Archaeology spike (timeboxed to 4 working days).** Establish what the inherited stack does: detection method, filter type and update rate, latency compensation scheme, serial protocol to the gimbal. Write it down. Do not refactor anything during this spike.
- **REQ-0.1** Structured logging of full estimator state at every update, not merely the final aim output. Logging the internals is what makes the error attributable rather than merely visible.
- **REQ-0.2** Gimbal encoder feedback (commanded vs. actual angle) logged on the same time base. This is nearly free in software and is the only way to separate estimator error from servo tracking error later.
- **REQ-0.3** Hardware frame trigger or strobe capture per §5.3, plus the LED verification test producing a measured sync-error figure.

Exit criterion: a recorded dataset exists, and the LED test reports a sync error ≤ 200 µs with the number written into the dataset metadata.

6.2 Phase 1 — Estimator measurement (Weeks 3–6)

Deliverable: the spinning rig itself, plus the analysis that turns a recording into the three numbers from goal G2.

- **REQ-1.1** A direct-drive spinning mount for a standard armor plate, holding commanded speed to within 2% at 0.25–3.0 rev/s, and also supporting a static hold mode and a step-change mode (sudden speed change) for measuring estimator convergence.
- **REQ-1.2** Absolute angular position of the plate available on the rig time base at ≥ 1 kHz with resolution ≤ 0.05°.
- **REQ-1.3** Analysis output: signed angular error vs. time, plus decomposition into constant bias (degrees), effective latency (milliseconds, estimated by cross-correlation lag between believed and true angle), and residual noise (RMS degrees), reported per spin rate.
- **REQ-1.4** The same analysis run against the gimbal feedback channel, so the error budget splits into "estimator got it wrong" vs. "gimbal did not go where it was told".

Exit criterion: a one-page report giving bias, latency and noise at 0.5, 1.5 and 3.0 rev/s, and at least one tuning decision made on the strength of it.

> **Hardware recommendation — spend nothing**
>
> Use a GM6020 gimbal motor as the spin actuator. It is direct drive, so there is no gearbox backlash between the encoder and the plate; it carries an absolute magnetic encoder at 8192 counts per revolution (0.044°, comfortably inside REQ-1.2); it is commanded over CAN; it reaches roughly 5 rev/s unloaded, covering the full required range; and the team almost certainly already owns spares.
>
> The alternative — an M3508 with its 19:1 planetary gearbox — is a trap. Its encoder sits on the rotor, upstream of the gearbox, so backlash makes the reading an unreliable proxy for plate angle and forces the purchase of a separate output-shaft encoder.

6.3 Phase 2 — Live-fire outcome baseline (Weeks 7–9)

This phase is deliberately downgraded from the original Stage 2. The referee system cannot do what the stage list implies.

Method	What it gives	What it cannot give	Cost
Referee system (0x0206 damage events)	Hit / no-hit count per magazine. Authoritative, matches competition scoring.	Impact location. ~200 ms link latency and ~3% packet loss mean a hit cannot be attributed to a spin angle — the target rotates 216° in that window at 3 rev/s. Armor detection also caps at 20 Hz, so rapid hits merge.	Requires a spare referee set — see Q2
Carbon / witness paper on the plate	Sub-millimetre impact positions for the whole group. Directly gives group centre offset and spread.	Per-shot timing, so a mark cannot be paired with the estimator state that produced it.	~$2
Piezo array (Phase 3)	Per-shot location AND timestamp, so each impact pairs with the estimator state at firing.	Accuracy realistically 15–25 mm, not millimetres.	~$40 + 3 weeks

REQ-2.1 — Measure the noise floor first. Before any spinning-target hit rate is recorded, fire a magazine at a static plate from a static, mechanically clamped launcher and measure the group spread on witness paper.

Projectile dispersion and muzzle-velocity variance set a hard floor on achievable accuracy. A hit-rate number recorded without knowing this floor is uninterpretable, and chasing an aimbot error smaller than the floor is wasted effort.

- **REQ-2.2** Log muzzle velocity per shot (available from the referee system) — it directly drives time-of-flight and therefore required lead angle, and is a plausible hidden source of misses.
- **REQ-2.3** Hit rate and group offset vs. spin rate, at fixed range, with witness paper photographed and digitised per magazine.

Exit criterion: a chart of hit rate vs. spin rate with the dispersion floor drawn on it as a horizontal reference.

6.4 Phase 3 — Piezo impact localisation (Weeks 10–14, go/no-go gated)

Four piezo elements bonded near the corners of the plate; an impact launches a flexural wave; the differences in arrival time across the four sensors locate the source by multilateration.

- **Feasibility.** Flexural (A0 Lamb) waves in a thin plate travel at roughly 1000–3000 m/s, so a wave crosses a 235 mm plate in about 80–120 µs. Resolving 10 mm needs timing precision of a few microseconds — trivially achieved by feeding four comparator outputs into an STM32 timer's input-capture channels, which resolve to nanoseconds. The clock is not the limitation.
- **The real limitation is physics, not electronics.** These waves are dispersive and they reverberate off plate edges, so the threshold-crossing instant shifts with impact energy and location. Published work on plate impact localisation reports errors of a few percent of plate dimension under good conditions. Accordingly the target is set at ≤ 25 mm RMS and ≥ 95% correct quadrant classification — not sub-centimetre. A design that promises millimetres here will fail its own acceptance test.
- **REQ-3.1** Go/no-go bench test in Week 10 before any integration effort: a spare plate, four sensors, and a set of controlled impacts at known grid positions. If calibrated RMS error exceeds 25 mm, stop and reallocate the remaining weeks to Phase 4.
- **REQ-3.2** Calibration procedure using a known grid of tap positions, with the resulting map stored alongside the rig configuration. Do not rely on an analytical wave-speed model; the empirical map will outperform it.
- **REQ-3.3** Each localised impact carries a rig-time-base timestamp so it can be joined to the estimator state at the moment of firing, minus time of flight. This pairing is the entire reason to build the array rather than use witness paper.

6.5 Phase 4 — Offline replay tuning (Weeks 6–8 partial, 15–16 consolidation)

This is the highest-value item in the project and the original plan buried it as a stretch goal. It should start as soon as Phase 1 produces its first dataset.

The insight: once a dataset contains camera frames plus hardware-aligned ground truth, the estimator can be re-run against that fixed recording with different parameters. Recording is expensive; replaying is nearly free. Parameter tuning becomes an offline optimisation over a static dataset, evaluated in seconds, with no motor spinning, no ammunition, and no robot.

- **REQ-4.1** A replay harness that reads a recorded dataset, runs the estimator with a supplied parameter set, and returns the error metrics from REQ-1.3 — deterministically, so the same inputs always give the same score.

- **REQ-4.2** A parameter sweep driver (grid or random search is sufficient; nothing more sophisticated is warranted at this dataset size) that ranks configurations by a defined scalar cost.
- **REQ-4.3** Held-out validation. Tune on one set of recordings, report the score on recordings never used during the search. Without this, the sweep will find parameters that memorise one dataset.
- **REQ-4.4** Any parameter set selected offline must be validated on the physical robot against a live spinning target before adoption. See risk R6.

7. Success metrics

#	Metric	Target	Measured how	By
M1	Ground-truth uncertainty	≤ 0.25° at 3 rev/s	LED sync test + encoder spec	Wk 3
M2	Estimator error characterised	Bias, latency, noise reported at 3 spin rates	Phase 1 report	Wk 6
M3	Tuning cycle time	≤ 10 min from parameter change to number	Stopwatch, 5 trials	Wk 8
M4	Real adoption	≥ 3 tuning decisions made on rig data	Decision log	Wk 10
M5	Measured improvement	Estimator error reduced ≥ 30% vs. the Week 6 baseline	Same rig, same protocol	Wk 14
M6	Live-fire baseline	Hit rate vs. spin rate curve with dispersion floor	Phase 2 chart	Wk 9
M7	Impact localisation (if Phase 3 proceeds)	≤ 25 mm RMS, ≥ 95% quadrant accuracy	Calibration grid holdout	Wk 14

M4 is the metric that matters most. A measurement tool that produces beautiful plots nobody acts on has failed, regardless of how accurate the plots are. If by Week 10 the rig has not changed a decision, the project is not working and §11 applies.

8. Technical requirements and bill of materials

8.1 Indicative BOM

Item	Purpose	Notes	Est. cost
GM6020 gimbal motor	Direct-drive spin + absolute encoder ground truth	Team almost certainly owns a spare; ~$110 if bought	$110
STM32F4 dev board	Rig time base, CAN, camera trigger, piezo input capture	F4-class timer resolution is ample	$15
USB-CAN adapter	Motor command + host link	Team likely has one	$25
24 V PSU	Motor supply	Or reuse a team battery	$25
Aluminium extrusion, base plate, fasteners	Rigid frame	Must not walk under vibration (risk R7)	$60
3D-printed hub + sensor mounts	Interface to standard armor plate	In-house, captive fasteners	$8
Opto-isolator, wiring, connectors	Camera trigger isolation		$12
LED + driver	Sync verification test		$8
Safety guard / polycarbonate shield	Spinning plate containment	Non-negotiable (risk R9)	$30

Item	Purpose	Notes	Est. cost
Carbon / witness paper	Phase 2 spatial ground truth	Highest value per dollar in the whole BOM	$5
4 × piezo discs + comparator front end	Phase 3 impact localisation	Do NOT order until the Week 10 go/no-go passes	$35
TOTAL (worst case: buy everything)		Inside the $500 ceiling with $167 to spare	$333
Committed before the Week 10 gate		Excludes likely-owned items and all Phase 3 parts	$163

Ordering the Phase 3 parts early would convert a reversible decision into a sunk cost, and sunk costs are exactly what make a solo builder integrate a sensor array that failed its own bench test.

8.2 Cross-cutting requirements

- **Safety.** The rig spins a rigid plate at up to 3 rev/s and is a live-fire target. Requirements: a mechanical guard or exclusion zone, an accessible emergency stop that cuts motor power in hardware rather than software, eye protection enforced whenever the launcher is armed, and captive fasteners on the plate hub. This is not optional and it is not overhead — a plate leaving the hub at speed is the most plausible way this project injures someone.
- **Data format.** Self-describing datasets: every recording carries its own metadata (rig configuration, measured sync error, camera settings, git commit of the aimbot under test, date). Datasets outlive code, and an unlabelled recording is worthless six months later.
- **Reproducibility.** Every reported number must be regenerable from a stored dataset by a single command. No numbers that exist only in a screenshot or a notebook cell.
- **Determinism.** The replay harness must be bit-for-bit deterministic. Non-determinism makes parameter comparison meaningless because run-to-run variance masks the effect being measured.

9. Risks

#	Risk	Impact	Likelihood	Mitigation
R1	Time synchronisation cannot reach ±200 µs (e.g. camera has no trigger or strobe line)	Fatal to premise — measurements become artefact	Medium	Confirm camera trigger capability in Week 1, before any other purchase. Fallback: cap spin rate at ~1 rev/s so the artefact stays below the signal, and state the reduced range explicitly in every report.
R2	Inherited stack is undocumented, unbuildable, or unloggable	Delays all phases; Phase 0 slips	High	Timeboxed 4-day archaeology spike with a written decision at the end: instrument it, or replace the tracker with a minimal known-good one. Do not drift into an open-ended refactor.
R3	Solo builder + live season → project starved of time	Project abandoned half-built	High	Phase gates deliver standalone value; Week 6 payback gate; explicit park criteria in §11. Prefer a finished Phase 1 over a half-finished Phase 3.
R4	No spare referee system available for the target	Phase 2 hit counting blocked	Medium	Witness paper covers the spatial measurement independently. Referee counting becomes optional rather than blocking. Confirm availability (Q2).
R5	Piezo localisation misses the 25 mm accuracy target	Phase 3 wasted	Medium	Week 10 bench go/no-go on a spare plate before any integration or full parts order. Reallocate to Phase 4 on a no-go.
R6	Overfitting to the rig — tuned parameters help against a spinning plate on a bench and hurt against a real robot	Silently degrades competition performance	Medium-High	The rig measures a proxy, not the goal. Mandate real-robot validation before adopting any tuned parameter (REQ-4.4); keep held-out datasets (REQ-4.3); vary lighting, range and background between recordings.
R7	Mechanical resonance or frame walk corrupts encoder truth at high spin	Silent bias in ground truth	Low-Medium	Sweep spin rate and look for anomalous encoder residuals; ballast or clamp the frame; record a static-plate control run in every session.
R8	Rig ground truth is trusted without validation	Confidently wrong conclusions	Medium	Independent check: film the plate with a phone at high frame rate and confirm the encoder-reported rate matches. Cheap, and it catches whole classes of wiring and unit errors.

R6 deserves particular attention. The moment a number exists, it becomes a target, and a solo builder optimising alone against a single bench setup is exactly the situation in which a proxy metric quietly diverges from the real objective. The rig is a diagnostic instrument, not a scoreboard.

10. Decisions and open questions

10.1 Decisions taken in this document

ID	Decision	Rationale
D1	Insert Phase 0 (observability + time base) ahead of all hardware work	Ground truth without sub-millisecond alignment measures its own artefact (§5)
D2	Use a direct-drive GM6020 and its absolute encoder as ground truth	No gearbox backlash; 0.044° resolution; already owned; removes a purchase
D3	Demote referee hit counting; promote witness paper for spatial data	~200 ms referee latency precludes angle attribution; paper is 400× cheaper and spatially better
D4	Promote offline replay tuning from stretch goal to Phase 4 starting Week 6	Highest value per hour in the project; gated only on Phase 0 datasets
D5	Set piezo accuracy target at ≤ 25 mm RMS, behind a Week 10 go/no-go	Wave dispersion and reverberation, not clock resolution, set the achievable limit

10.2 Open questions — needed before Week 1 closes

ID	Question	Blocks	Needed by
Q1	What camera does the aimbot use, and does it expose an external trigger input or a strobe output?	REQ-0.3, the entire time-base design, and the usable spin-rate range	Week 1
Q2	Is a spare referee system set available to mount on the target rig?	Phase 2 hit counting (not spatial measurement)	Week 6
Q3	Is there a space where the launcher can be fired safely and repeatedly at a fixed range?	All of Phase 2; may force a shorter test range	Week 6
Q4	What compute runs the aimbot, and does it have spare capacity and disk throughput for full-rate logging?	REQ-0.1 — logging that drops frames under load produces biased datasets	Week 1
Q5	Which armor plate size is the reference target — small (~135 mm) or large (~235 mm)?	Hub design, piezo geometry, calibration grid	Week 2
Q6	Will anyone else on the team use this, even informally?	Whether any documentation effort is justified	Week 8

11. Kill and park criteria

Written now, while judgement is uncommitted, because a solo builder four months into a project is the worst possible judge of whether to continue.

- **Week 3 — hard gate.** If measured sync error exceeds 1 ms and no hardware path to improve it exists, stop. Reduce scope to a static-target accuracy check, which is still useful and costs almost nothing, and abandon the spinning rig.
- **Week 6 — payback gate.** If Phase 1 has not produced a number that changed a tuning decision, park the project until the off-season. Robot work wins during the season.

- **Week 10 — Phase 3 gate.** If the piezo bench test misses 25 mm RMS, do not integrate. Reallocate the time to Phase 4.
- **Any time — scope tripwire.** If more than two consecutive weeks pass with no dataset recorded, the project has drifted into building infrastructure for its own sake. Record something, however crude, and re-plan.

Companion artefact: the phased roadmap workbook, containing the week-by-week plan with dependencies and exit criteria, the BOM with a budget check, the risk register, and the open-questions tracker.